

Manitas *MX* 🤞

Traductor de Lengua de Señas Mexicana con Inteligencia Artificial para iPhone y iPad

Reconocimiento en tiempo real del abecedario dactilológico LSM

Institución

UNAM — Facultad de Ingeniería

Profesor

Ing. Marduk Pérez de Lara Domínguez

Curso

Cómputo Móvil · Grupo 03 · 2026-2

Fecha de entrega

29 de mayo de 2026

Equipo: Code & Coffee

Cruz Buenavista Lesliee Sarahí

Martínez Bravo Daniela

Pelaez Semprun Diego Eduardo



01— Requerimientos

Manitas MX combina captura de cámara, análisis de postura de mano con Apple Vision Framework y clasificación semántica mediante Gemini IA para reconocer el abecedario dactilológico LSM.

Reglas de negocio:

- Solo reconoce letras individuales del abecedario LSM, incluyendo la Ñ
- No reconoce palabras, frases ni conversaciones en el MVP
- El usuario debe otorgar permiso de cámara
- La detección requiere al menos una mano visible
- La posición debe mantenerse estable unos segundos
- No se almacenan imágenes ni video de forma permanente
- El historial guarda letra, hora y nivel de confianza
- Se puede borrar registros individuales o todo el historial
- Si la confianza es baja, se muestra estado "no identificada"

Requerimientos funcionales:

- Pantalla inicial con tutorial de uso
- Solicitar permiso de cámara antes de la detección
- Activar cámara frontal o trasera
- Permitir cambio entre cámaras
- Detectar postura de mano con Vision Framework
- Extraer descripción espacial de la mano y dedos
- Enviar descripción e imagen temporal a Gemini
- Mostrar letra detectada y nivel de confianza
- Registrar detecciones en historial
- Consultar abecedario LSM con imagen y descripción
- Borrar registros con deslizamiento o botón

Requerimientos no funcionales:

- Usabilidad: interfaz sencilla, intuitiva y en español
- Accesibilidad: tipografía clara y contraste visual adecuado
- Privacidad: no almacena imágenes ni video
- Seguridad: las API keys no se exponen en el código fuente
- Rendimiento: respuesta en pocos segundos, navegación fluida
- Compatibilidad: iPhone y iPad con iOS/iPadOS 16 o superior
- Mantenimiento: código organizado en módulos

02 — Viabilidad técnica

Análisis de qué es posible construir dentro del alcance del MVP.

✓ Funcionalidad propuesta

El reconocimiento de letras del abecedario dactilológico LSM mediante cámara, visión artificial e IA generativa es la funcionalidad técnica central. Captura la mano del usuario, analiza su postura y utiliza Gemini 2.5 Flash para interpretar la configuración de los dedos y clasificar la letra correspondiente.

- ✓ Letras A–Z
- ✓ Letra Ñ
- ✓ Detección en tiempo real

Justificación de viabilidad



Viable:

reconocimiento de letras A–Z + Ñ del abecedario dactilológico LSM



No viable en MVP:

reconocimiento de palabras o frases completas (requiere análisis temporal y dataset más extenso)



No viable en MVP:

funcionamiento completamente offline (requeriría un modelo de IA local en el dispositivo)

03 — Alcance

Alcance general

Aplicación para iPhone y iPad con tres módulos principales: detección en vivo, aprendizaje y consulta del alfabeto LSM, e historial de traducciones controlado por el usuario. Uso de IA para clasificación de letras, interfaz en español.

- Detección en vivo
- Aprendizaje LSM
- Historial
- Gemini API

Fuera del alcance inicial

- Reconocimiento de palabras completas y frases
- Cuentas de usuario
- Funcionamiento offline
- Reconocimiento de conversaciones completas
- Versión para otros dispositivos (Android, web)

MVP — Producto Mínimo Viable

El MVP implementa los módulos de cámara, enseñanza e historial, con los requerimientos de privacidad y permisos del usuario, así como la conexión mediante API a Gemini.

-  Módulo de cámara
-  Módulo de enseñanza
-  Historial
-  Privacidad y permisos
-  Gemini API



04 — Wireframes

- Pantallas principales en el orden real del flujo de navegación, con documentación de funcionalidad, datos, tipo, vigencia y operaciones por pantalla.

Manitas MX - Traductor LSM



Permisos de cámara

Primera pantalla. Solicita autorización para usar la cámara del dispositivo.

(Bool, String | Temporal | Consulta + Registro)



Bienvenida

Presenta el nombre, objetivo general y un botón para comenzar.

(String | Estático | Bundle local)



Detección

Cámara activa con área de detección. El usuario coloca la mano para traducir la seña en tiempo real.

(CVPixelBuffer, Bool, String | Temporal/frame | Consulta + Actualización)



Aprendizaje

Lista ordenada del abecedario LSM (A–Z y Ñ) con descripción breve por letra.

(Array, String | Estático | Consulta + Filtrado)



Detalle de letra

Explicación completa de una letra: forma, consejo clave y botón "Practicar".

(String, UIImage | Estático | Bundle local)



Historial

Letras detectadas con hora y nivel de confianza. Permite borrado individual y total.

(Array, Date, Float | Persistente | Consulta + Borrado)



Privacidad

Informa cómo se manejan los datos capturados y qué se conserva localmente.

(String | Estático | Solo lectura)



Detección no identificada

Se muestra cuando no hay mano visible o la confianza de clasificación es baja.

(String, Float | Temporal | Consulta + Actualización)



Permisos IA

Solicita autorización para procesar la imagen mediante inteligencia artificial.

(Bool, String | Temporal | Consulta + Registro)



Analizando letra

Indicador de carga mientras Vision Framework y Gemini procesan la letra capturada.

(String, Float | Temporal | Consulta + Actualización)

04 — Wireframes

Manitas MX - Traductor LSM

- Pantallas principales en el orden real del flujo de navegación, con documentación de funcionalidad, datos, tipo, vigencia y operaciones por pantalla.

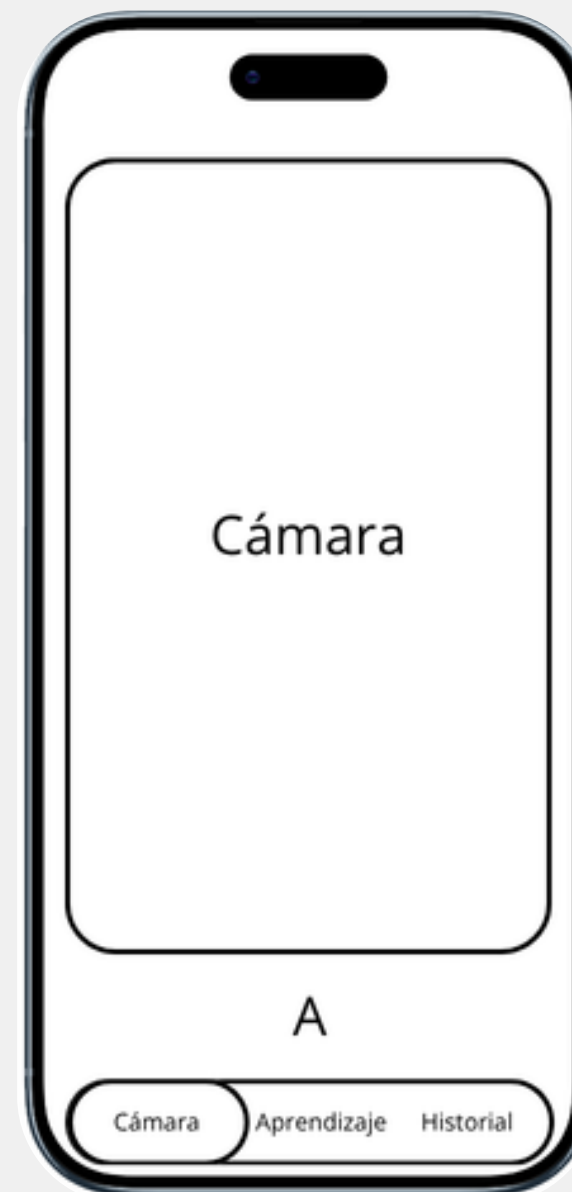
 **Permisos de cámara**



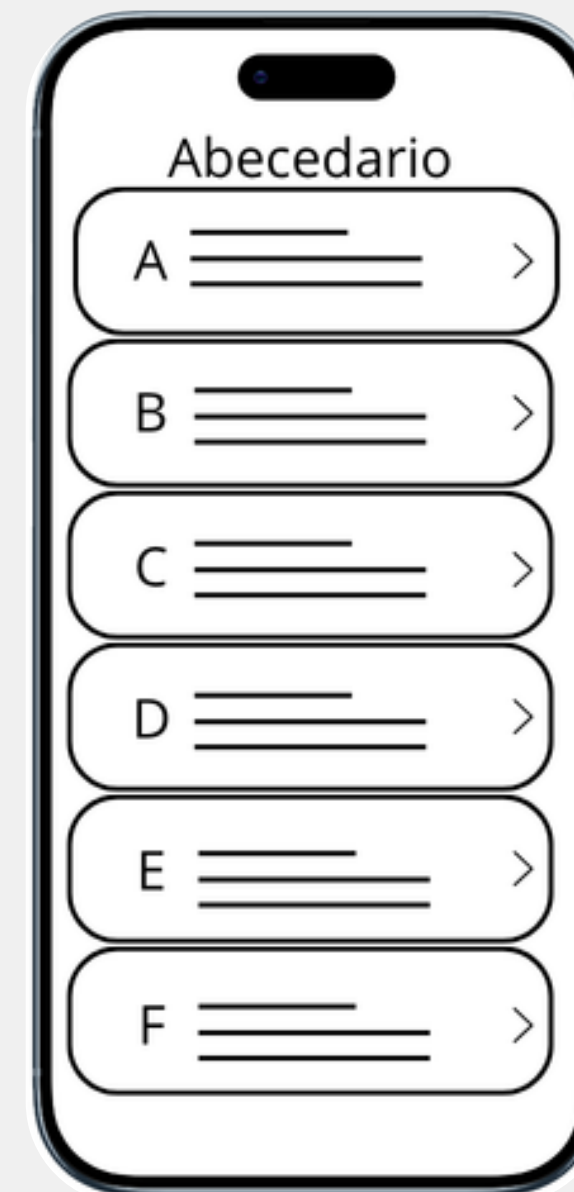
 **Bienvenida**



 **Detección**



 **Aprendizaje**



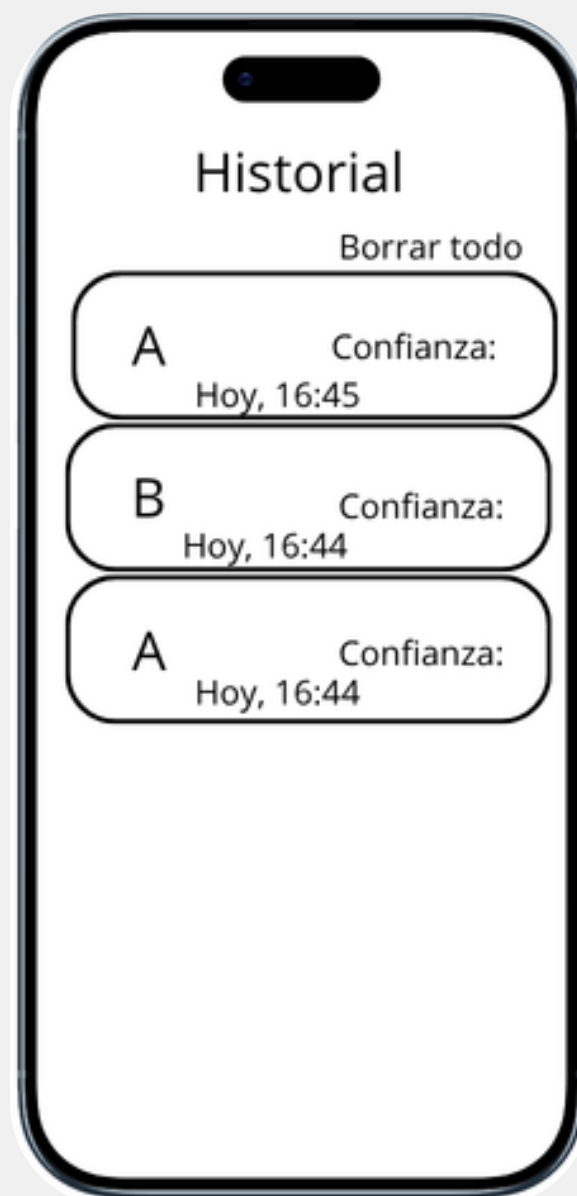
 **Detalle de letra**



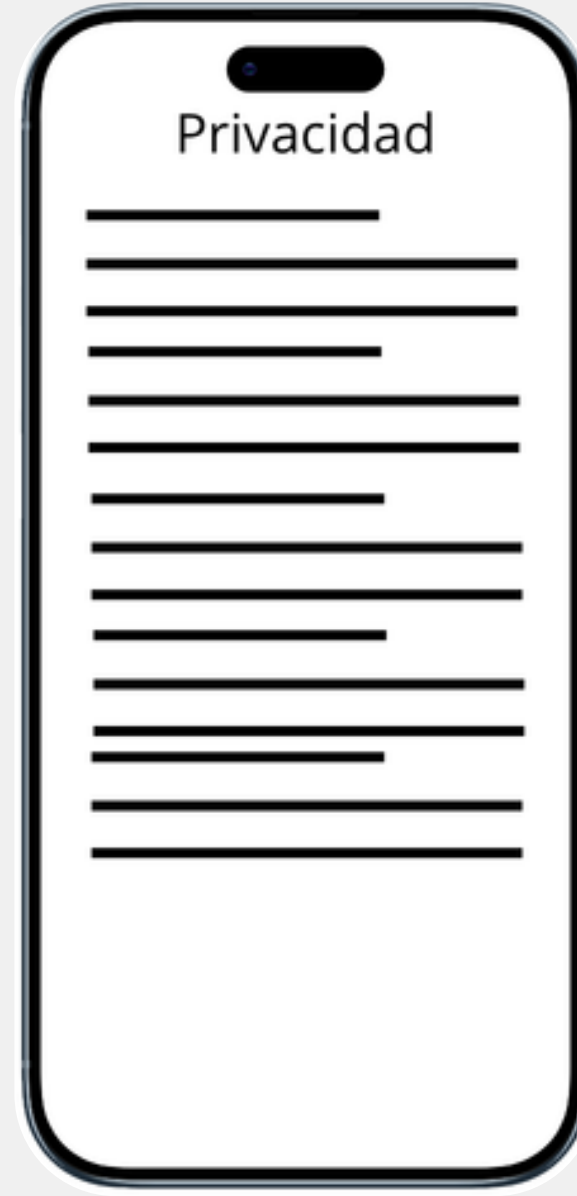
04 — Wireframes

- Pantallas principales en el orden real del flujo de navegación, con documentación de funcionalidad, datos, tipo, vigencia y operaciones por pantalla.

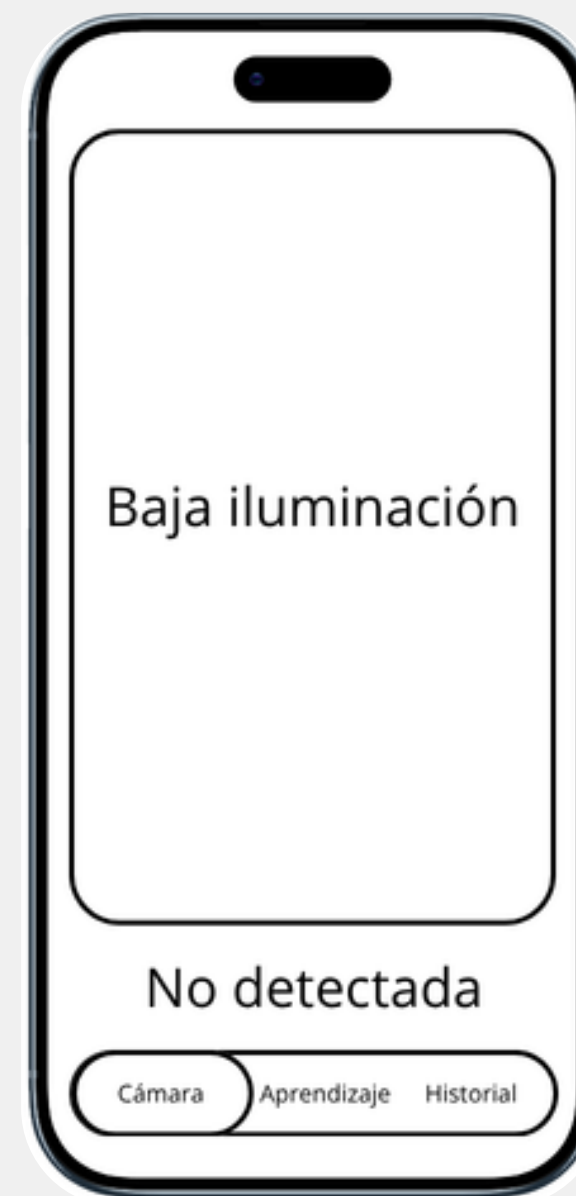
 **Historial**



 **Privacidad**



 **Detección no identificada**



 **Permisos IA**



 **Analizando letra**



05 — Flujos

Manitas MX - Traductor LSM

Flujo principal



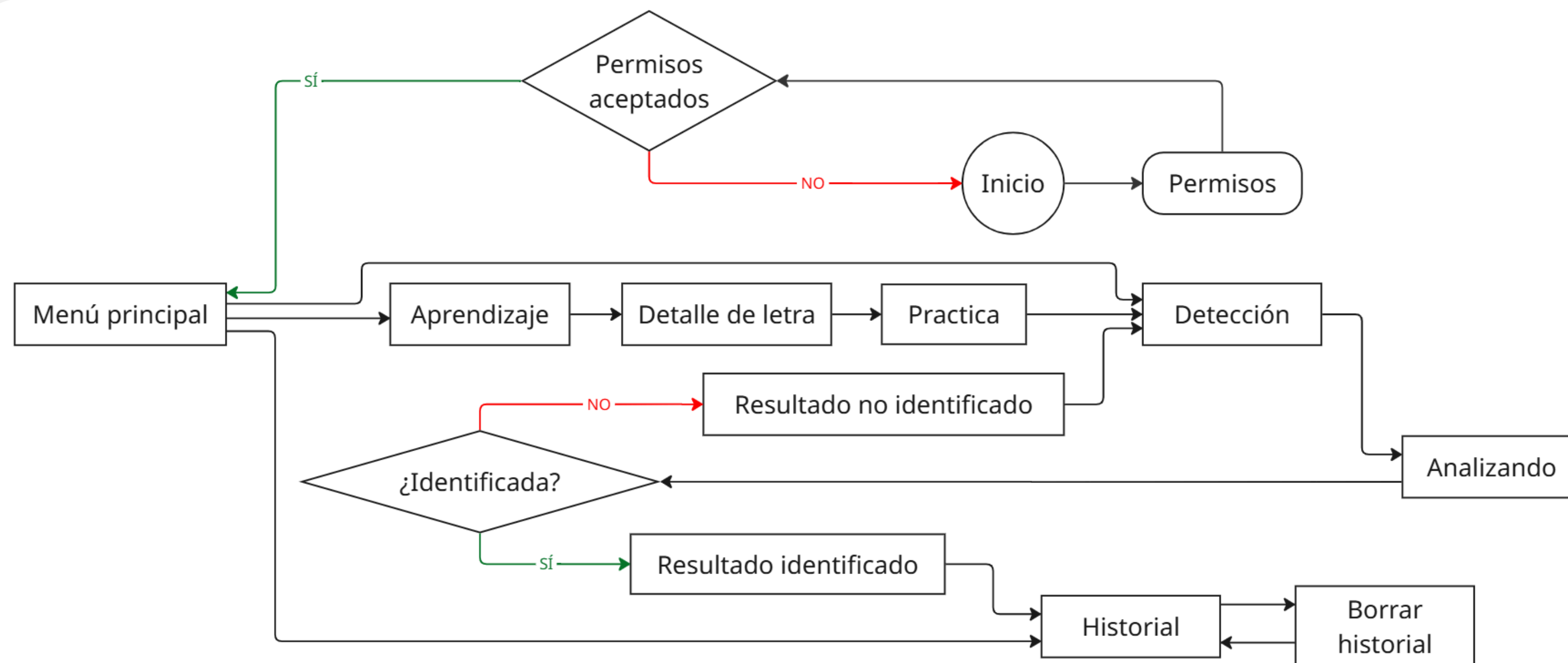
Flujo cíclico Aprendizaje → Práctica

Desde el módulo de aprendizaje, el usuario puede consultar una letra y pulsar "Practicar" para ir directamente al módulo de detección e intentar recrearla en tiempo real. Esto conecta el contenido educativo con la práctica inmediata.

Navegación inferior (TabView)

La barra de navegación inferior de SwiftUI permite moverse entre los tres módulos principales en cualquier momento: Detección, Aprendizaje e Historial. El historial admite borrado por deslizamiento o mediante botón.

06 — Cambios de estado

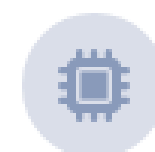


07 — Dispositivos y sensores



Dispositivos y orientaciones

- ✓ iPhone y iPad con iOS / iPadOS 16 o superior
- ✓ Diseñada principalmente para orientación vertical (facilita uso con una mano)
- ✓ Compatible con orientación horizontal sin afectar rendimiento ni usabilidad

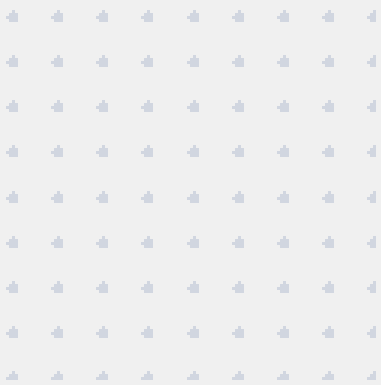


Sensores utilizados

-  Cámara frontal y trasera: captura de video para detectar la mano y las letras LSM
 -  Sensor de orientación: permite el cambio de orientación de la pantalla
-
-  **No se usan:** ubicación, micrófono, Bluetooth ni otros sensores

08 — Demo

Manitas MX – Traductor LSM



Las pantallas incluidas muestran la paleta de colores, tipografía, disposición de componentes y estructura de navegación de Manitas MX.



Pantalla de bienvenida

Nombre de la app, objetivo general y botón "Comenzar". Punto de entrada al flujo principal.



Módulo de aprendizaje

Lista del abecedario LSM completo (A–Z, Ñ) con imagen y descripción para cada letra.



Historial de detecciones

Registro de letras traducidas con hora y confianza. Borrado por deslizamiento o por lote.



Detección en tiempo real

Vista de cámara activa, área de captura y letra resultante con nivel de confianza en pantalla.

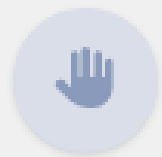
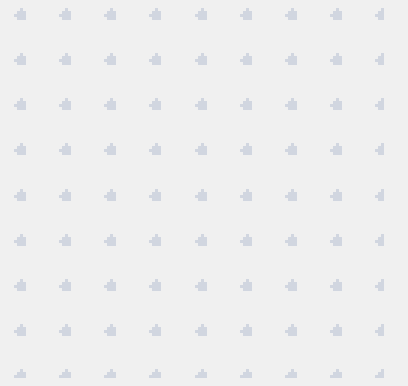


Detalle de letra LSM

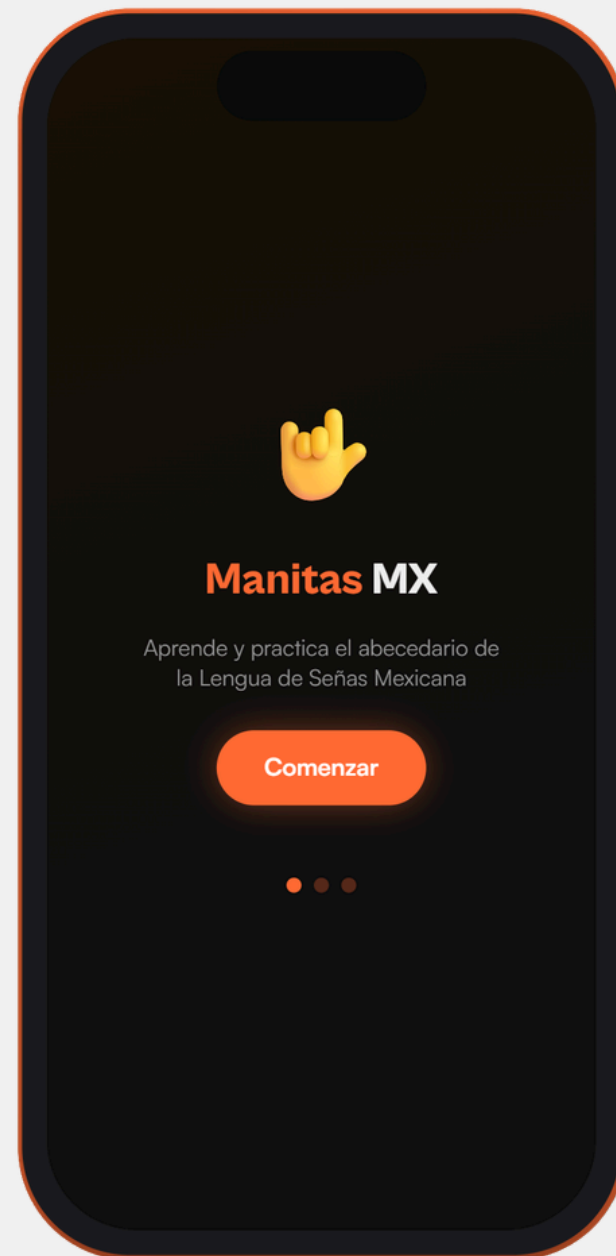
Explicación técnica completa, imagen de referencia, consejos y botón "Practicar".

08 — Demo

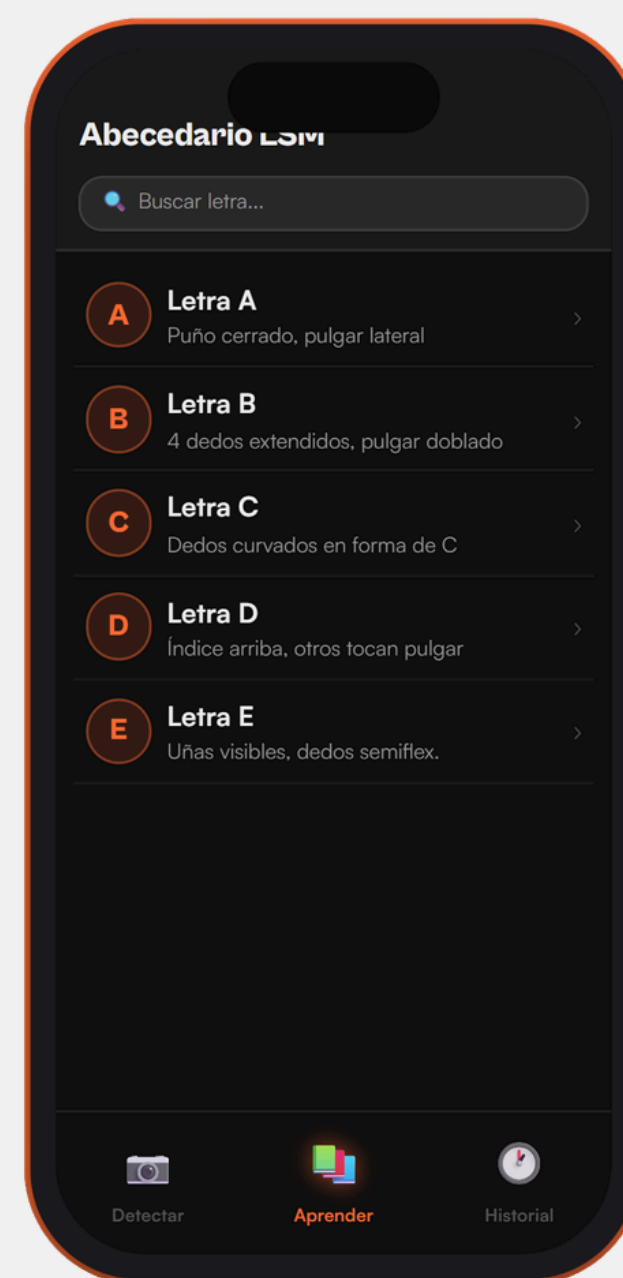
Manitas MX – Traductor LSM



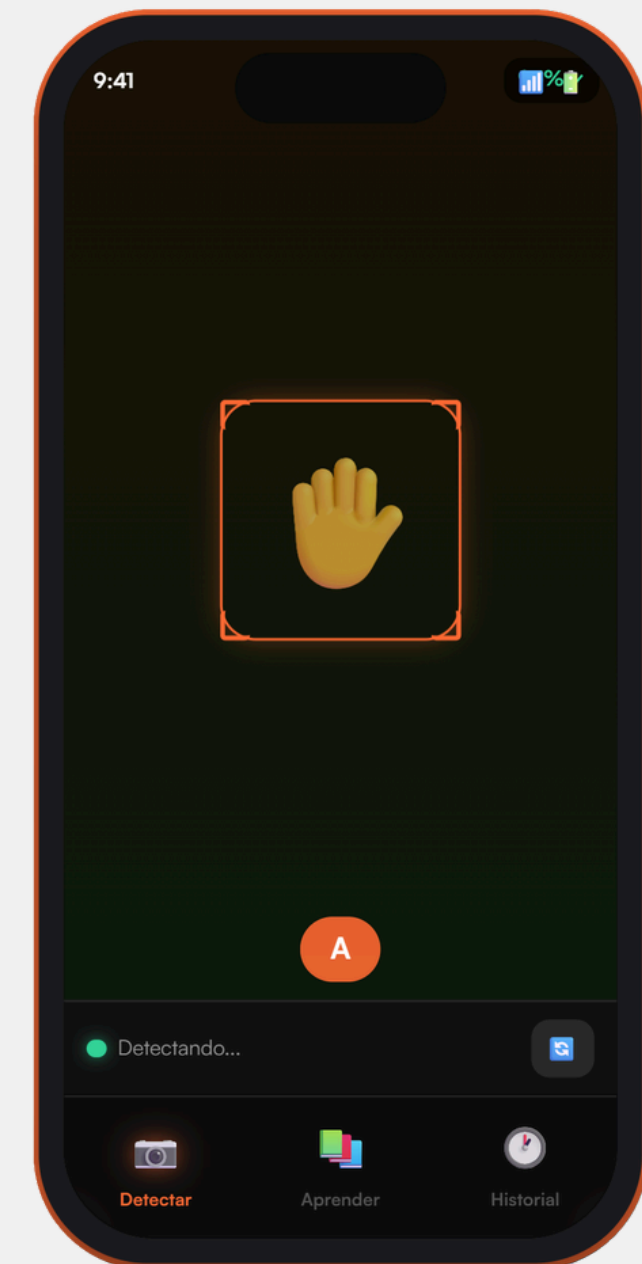
Pantalla de bienvenida



Módulo de aprendizaje

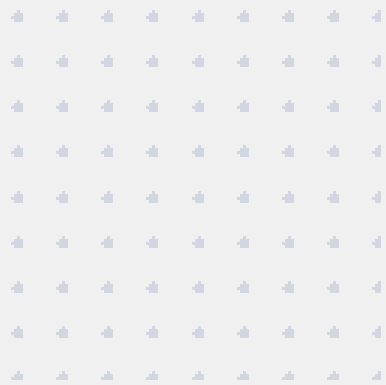


Detección en tiempo real



08 — Demo

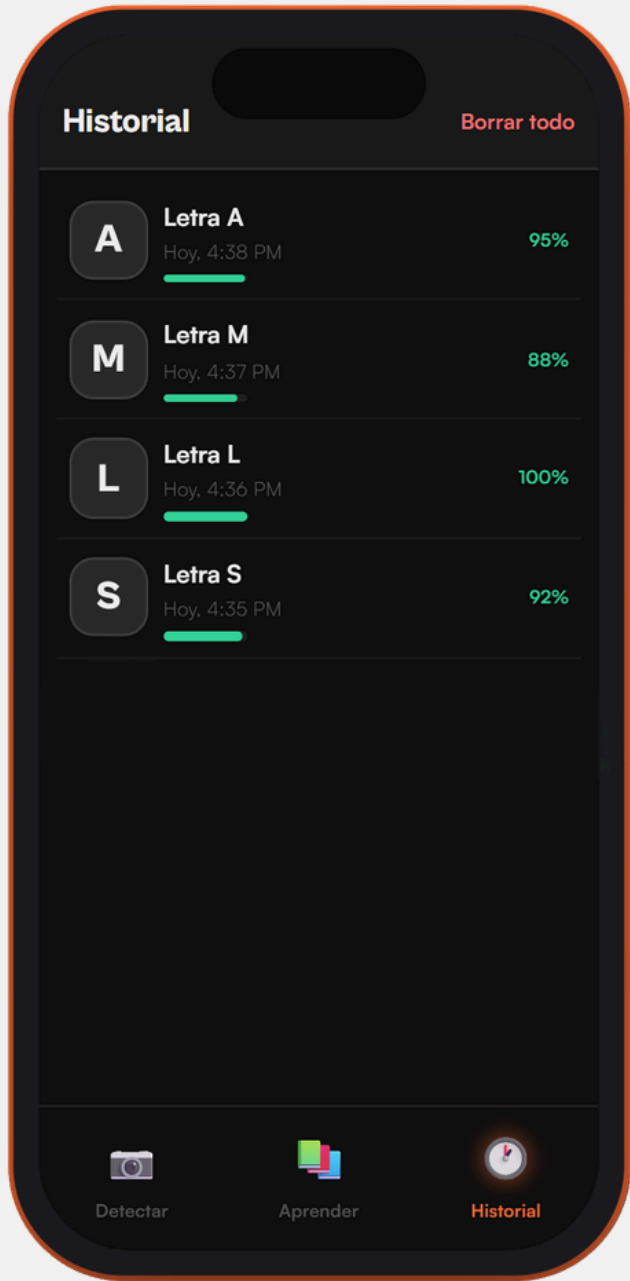
Manitas MX – Traductor LSM



Detalle de letra LSM



Historial de detecciones



09 — Lenguajes, herramientas y frameworks

Lenguajes de programación

- **Swift:** lenguaje principal para el desarrollo nativo en iOS
- **JSON:** formato de intercambio de datos para solicitudes y respuestas de la API de Gemini

Herramientas

- **Xcode:** desarrollo, compilación, ejecución y prueba en dispositivo físico
- **GitHub:** control de versiones, colaboración y respaldo del código
- **Google Docs:** redacción colaborativa del reporte
- **Inteligencia artificial:** depuración, comprensión de errores y documentación

Frameworks

- **SwiftUI:** interfaz declarativa, vistas, navegación, TabView y componentes reutilizables
- **AVFoundation:** configuración de cámara, captura de video y cambio entre cámaras
- **Vision Framework:** detección de postura de mano y extracción de puntos de referencia

Gemini 2.5 Flash — Servicio externo

Único servicio externo. Se usa exclusivamente en el módulo de detección.

Característica	Detalle
Proveedor	Google DeepMind — Gemini 2.5 Flash
Endpoint	REST API — generativelanguage.googleapis.com
Formato	JSON. Request: descripción geométrica + imagen temporal. Response: letra, confianza, descripción.
Autenticación	API Key por header. MVP: pool de keys rotativas. Producción: backend con tokens temporales.
Costo	Variable por tokens. Texto: bajo costo. Imagen: alto consumo (problema identificado). Se mitiga con caché.
Disponibilidad	Requiere conexión a internet. Sin fallback offline en MVP.
Optimización	Sistema de caché que evita llamadas cuando la postura no ha cambiado significativamente.
Seguridad (prod.)	Las API keys deben moverse a un backend que emita tokens temporales.

10 — Arquitectura y metodología

Manitas MX - Traductor LSM



Arquitectura MVVM adaptada

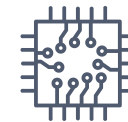


Patrones de diseño



Presentation

Vistas y pantallas de SwiftUI



MVVM

Separación de vistas, modelos y estado observable



Domain

Reglas de negocio y modelos



Observer / ObservableObject

Actualización de interfaz ante cambios de estado



Data

Datos de la app: historial y detecciones



Singleton controlado

Pool de API keys y caché de resultados compartidos



Infrastructure

Manejo de sistemas: cámara, Vision Framework



Componentización

Vistas reutilizables: botones, tarjetas, indicadores, overlays

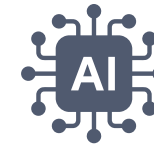
 *Nota: En el MVP no existe backend propio para las API keys. Se recomienda agregarlo en la versión final para controlar cuotas y mejorar la seguridad.*

10 — Arquitectura y metodología



Metodología de desarrollo

- Prototipado rápido e iterativo (adecuado para hackathon)
- Definición del problema, alcance mínimo viable y público objetivo
- Determinación de módulos: cámara, detección, aprendizaje, historial e interfaz
- Primera versión funcional mínima con iteraciones de mejora del prompt
- Corrección de errores, ajustes de cámara, interfaz y optimización de API keys en cada iteración



Uso de inteligencia artificial

- **Dentro de la app:** Gemini clasifica letras LSM con instrucciones estrictas de salida para mostrar la traducción en tiempo real
- **Durante el desarrollo:** comprensión de Swift y SwiftUI, estructurar prompts, depurar errores, documentación y decisiones técnicas.

11 – Administración del proyecto



Fases del proyecto

- Investigación de la problemática
- Definición del alcance
- Diseño de interfaz y funcionalidades
- Desarrollo del prototipo
- Integración con inteligencia artificial
- Pruebas técnicas y de usabilidad
- Documentación y entrega final



Herramientas de administración

- **GitHub:** compartir código, respaldar y entrega final
- **Google Docs:** edición colaborativa de la documentación
- **WhatsApp:** comunicación entre todos los integrantes del equipo

Equipo de trabajo y roles

Integrante	Participación principal	Apoyo complementario
Cruz Buenavista Lesliee Sarahí	Redacción y organización de la documentación, integración conceptual de requerimientos	Revisión general del reporte, apoyo en la presentación y en la definición del alcance
Martínez Bravo Daniela	Diseño de interfaz, estructuración de pantallas, wireframes y propuesta visual	Apoyo en documentación, revisión de pantallas y preparación de la presentación
Pelaez Semprun Diego Eduardo	Desarrollo técnico, lógica de detección e integración con Gemini	Apoyo en documentación, presentación y resolución de errores con el equipo

Nota: El trabajo fue colaborativo. La documentación se dividió por secciones y se unificó entre todos. El desarrollo técnico se realizó en conjunto, especialmente en implementación, pruebas y corrección de errores.

12 — Permisos y publicación

Permisos requeridos

- **Cámara:** necesaria para detectar y traducir señas en tiempo real
- En el MVP no se requiere acceso a ubicación, micrófono, contactos, galería de fotos ni Bluetooth

Adecuaciones para publicación

- **API keys:** eliminar del código fuente y usar un sistema de tokens temporales (tiene costo monetario elevado)
- **Política de privacidad:** indicar uso de cámara, envío de imagen a servicio externo y almacenamiento local del historial
- **Consentimiento de IA:** agregar manejo explícito de consentimiento para procesamiento de imagen mediante inteligencia artificial

13 — Costos y tiempo de desarrollo

Manitas MX - Traductor LSM



Tiempo estimado de desarrollo

- **2 días**
Prototipo inicial del hackathon
- **2 semanas**
Ajuste del MVP
- **2 semanas**
Pruebas técnicas, corrección de errores y mejora del prompt
- **3 semanas**
Pruebas con usuarios reales y asesoría de expertos
- **1 semana**
Preparación para publicación (aviso de privacidad, materiales App Store, revisión de permisos)
- **≈ 8 semanas y 2 días**
Total estimado

Costos de desarrollo

- **Contexto académico (equipo Code & Coffee):** \$0 MXN
- **Implementación profesional completa:** Variable (incluiría horas de programación, análisis, arquitectura de software, diseño UX, pruebas de calidad y coordinación de equipo)

Costos de implementación

- **Cuenta Apple Developer:** \$99 USD/año
- **Gemini API:** Variable por uso
- **Backend propio:** Variable

Costos de mantenimiento

- Renovación Apple Developer: \$99 USD/año

14 — Referencias



1. Secretaría de Salud. (2021). 530. Con discapacidad auditiva, 2.3 millones de personas. Gobierno de México.
2. CNDH. (2012). Norma Oficial Mexicana NOM-031-SSA3-2012. Diario Oficial de la Federación.
3. Toolify. (s.f.). IA centrada en el ser humano: Guía completa y beneficios. Toolify AI News ES.
4. SEP. (2012). Orientaciones para la atención educativa de alumnos sordos. Modelo educativo bilingüe-bicultural.
5. Ramírez, A. (2025). Lengua de señas mexicana. Incluyeme.com.
6. Ágora UNIVA. (2023). Lengua de Señas Mexicana, ¿por qué es importante? Universidad del Valle de Atemajac.
7. Jiménez Ortega, M. J. (2024). Importante, enseñanza de Lengua de Señas Mexicana. Boletines UAM, (463).
8. INEGI. (2017). Encuesta Nacional sobre Discriminación (ENADIS) 2017: Principales resultados.
9. CONAPRED. (2023). Ficha temática: Discriminación en contra de las personas con discapacidad.
10. Ley General para la Inclusión de las Personas con Discapacidad. DOF, 30 de mayo de 2011 (México). Última reforma 14 de junio de 2024.
11. Microsoft. (2023). AI for Accessibility Impact Report 2023. Microsoft Corporation.
12. AQMUN. (2024). Reformas educativas para la inclusión de la comunidad sorda en América Latina. Universidad Anáhuac Querétaro.



Cruz Buenavista · Martínez Bravo · Pelaez Semprun

Equipo Code & Coffee · UNAM Facultad de Ingeniería · Cómputo Móvil 2026-2